

Agentic AI: Foundations and Open-Source Practice

Day 1 Study Material

From LLMs to Agents, and Agents vs Chatbots vs Workflows

1. Session Overview

Unit	Unit 1: Foundations of AI Agents
Sub-topics	1.1 From LLMs to agents 1.2 Agents vs chatbots vs rule-based workflows
Course Outcome	CO1

Learning Objectives

By the end of Day 1, you will be able to:

1. Explain what a large language model (LLM) can and cannot do on its own.
2. Describe an AI agent as the combination of an LLM, tools, and a loop.
3. Define autonomy and agency, and place a system on the autonomy spectrum.
4. Compare chatbots, rule-based workflows, and agents on autonomy, reasoning, tool use, and determinism.
5. Decide when a workflow is a better engineering choice than an agent.

2. Video Lectures

Watch Videos 1 and 2 before class. Videos 3 and 4 explain the comparison between assistants and agents, Video 5 is optional enrichment, and Video 6 helps you install the tools used in the labs.

No.	Title and Link	Channel	When and Why
1	AI Agents, Clearly Explained https://www.youtube.com/watch?v=FwOTs4UxQS4	Jeff Su	Watch before class (required). Walks from plain LLMs to AI workflows to AI agents using everyday examples. Maps directly to sub-topics 1.1 and 1.2.
2	What are AI Agents? https://www.youtube.com/watch?v=F8NKVhkZZWI	IBM Technology	Watch before class (required). Explains compound AI systems, the move from fixed program logic to LLM-controlled logic, and why agents need tools and memory.
3	AI Agents and AI Assistants: A Contrast in Function https://www.youtube.com/watch?v=livxYYkJ2DI	IBM Technology	Recommended. Contrasts reactive assistants (chatbots) with proactive agents; supports sub-topic 1.2.
4	Generative vs Agentic AI: Shaping the Future of AI Collaboration https://www.youtube.com/watch?v=EDb37y_MhRw	IBM Technology	Recommended. Distinguishes content-generating AI from goal-pursuing AI.
5	What's next for AI agentic workflows ft. Andrew Ng	Sequoia Capital	Optional enrichment (about 14 minutes). Shows why iterative agentic workflows can beat a

No.	Title and Link	Channel	When and Why
	https://www.youtube.com/watch?v=sal78ACtGTc		stronger model used in a single pass, and previews the design patterns for Day 2.
6	Ollama Course – Build AI Apps Locally https://www.youtube.com/watch?v=GWB9ApTPTv4	freeCodeCamp.org	Setup help (watch only the setup and basic command sections). The full course is about 3 hours; the remaining parts are useful for Day 5.

If a link does not open, search the exact video title on YouTube.

3. Lecture Notes: 1.1 From LLMs to Agents

3.1 What is a Large Language Model?

A large language model is a neural network (a Transformer) trained on a very large amount of text to predict the next token. A token is a small piece of text, roughly a word or part of a word. When you send a prompt, the model repeatedly predicts the most suitable next token until the answer is complete.

Everything an LLM "knows" comes from two places: the patterns it learned during training, and the text you place in its input (the **context window**). It has no other source of information.

3.2 What an LLM Can Do on Its Own

- Understand and generate natural language: summaries, emails, explanations, translations.
- Follow instructions and adopt a role or tone.
- Reason step by step over information given in the prompt.
- Write and explain code.
- Choose between options when the options are described to it. This ability is what makes agents possible.

3.3 What an LLM Cannot Do on Its Own

Limitation	Why it happens	Classroom example
No private data	It was never trained on your organization's data.	It cannot know a student's attendance or a course's fee.
No current information	Training stops at a knowledge cutoff date.	It cannot tell today's weather or this week's exam timetable.
Cannot take actions	It only produces text; it cannot run programs, send emails, or update databases.	It can draft a leave letter but cannot submit it.
Unreliable arithmetic	It predicts digits as text rather than computing them.	Multiplying two large numbers may give a wrong answer.
Hallucination	When information is missing, it still produces fluent, confident text.	It may invent a fee amount that sounds realistic.

Limitation	Why it happens	Classroom example
Stateless	Each API call is independent; it remembers nothing unless previous messages are sent again.	A new chat does not remember yesterday's conversation.
Passive	It waits for a prompt and responds once; it does not pursue a goal over several steps.	It will not check its own answer unless told to.

3.4 From LLM to Agent: Agent = LLM + Tools + Loop

An AI agent is a system in which an LLM decides, step by step, which actions to take to achieve a goal. Each missing capability above is filled by adding a component around the LLM:

Component	Role	Fixes which limitation
LLM	The reasoning engine. Reads the goal and the history, and decides what to do next.	(Provides reasoning)
Tools	Functions the LLM can ask the program to run: database lookup, calculator, web search, send email.	Private data, current information, actions, arithmetic
Loop	The program keeps calling the LLM, running requested tools, and feeding results back until the task is done.	Passivity; enables multi-step work and self-correction
Memory (Day 2 and Day 9)	Conversation history and stored facts carried across steps.	Statelessness

The agent loop in five steps:

1. **Goal:** the user gives a task, for example "total fee for CS101 and AI202 after a 10% scholarship".
2. **Reason:** the LLM decides what it needs: the fee for CS101.
3. **Act:** the LLM requests a tool call, `get_course_fee("CS101")`; the program runs it.
4. **Observe:** the result (12000) is added back into the conversation.
5. **Repeat or finish:** the LLM decides whether more steps are needed (AI202 fee, then calculator) or whether it can give the final answer.

Key idea: the LLM never runs the tool itself. It only outputs a structured request (tool name + arguments). Your Python program executes the tool and returns the result. This separation is what keeps agents controllable.

3.5 Autonomy and Agency

Agency is the ability of a system to act on its environment in pursuit of a goal (for example, calling tools that read or change data). **Autonomy** is how much the system decides for itself without a human choosing each step. The two are related but different: a system can have a lot of agency (it can send emails) but low autonomy (a human approves each email).

It helps to think of autonomy as a spectrum rather than a yes/no property:

Level	Name	Who decides the next step
0	Traditional software	The programmer, entirely in code. No LLM.
1	LLM chatbot	A human asks; the LLM answers once. No tools, no loop.
2	LLM workflow (chain)	The programmer fixes the sequence of steps; the LLM fills in individual steps (e.g., summarize, then translate).
3	Router / tool-calling	The LLM chooses one path or tool from a fixed set, once.
4	Agent	The LLM chooses tools repeatedly in a loop and decides when the goal is reached.
5	Multi-agent / fully autonomous	Several agents plan, delegate, and act over long periods with little human involvement (Unit 6).

Engineering guidance: use the **lowest level of autonomy that solves the problem**. Each step up the spectrum adds flexibility but also cost, latency, and unpredictability.

4. Lecture Notes: 1.2 Agents vs Chatbots vs Rule-Based Workflows

4.1 Definitions

Traditional chatbot: matches user messages to predefined intents and replies with scripted responses (for example, a bank's menu-driven chatbot). **LLM chatbot:** generates free-form answers with an LLM, but only from its training and the conversation; it takes no actions.

Rule-based workflow: a program with fixed, human-written control logic (if/else, fixed sequences). It may call an LLM at certain steps, but the path through the steps is decided by the programmer in advance.

AI agent: an LLM controls the path. It chooses which tools to use, in what order, and how many times, based on the goal and the results it observes.

4.2 Comparison

Dimension	Chatbot (LLM)	Rule-based workflow	AI agent
Autonomy	Low: responds once per message	None: path fixed by programmer	High: decides its own steps
Reasoning	Within one reply only	None in control flow (logic is hard-coded)	Across multiple steps; plans and revises
Tool use	None	Fixed tools called in a fixed order	Chooses tools dynamically
Determinism	Varies between runs	Fully deterministic: same input, same output	Least predictable: path can change between runs
Handles new question types	Yes in language, but may hallucinate facts	No: fails outside its rules	Yes, if suitable tools exist

Dimension	Chatbot (LLM)	Rule-based workflow	AI agent
Cost and latency	One LLM call	Very low (or one call per fixed step)	Many LLM calls per task
Debugging and testing	Moderate	Easy: trace the code	Hard: must inspect the reasoning trace
Typical example	FAQ assistant, writing helper	Fee receipt generation, form validation, scheduled reports	Research assistant, IT support agent, coding agent

4.3 When a Workflow Is Better Than an Agent

Agents are not always the right answer. Choose a **workflow** when most of these statements are true:

- The steps are known in advance and rarely change.
- The same input must always produce the same output (audits, finance, grading, compliance).
- Errors are costly and every step must be traceable.
- Speed and low cost matter more than flexibility.
- The input types are limited and well defined.

Choose an **agent** when:

- The task is open-ended and the number or order of steps cannot be predicted.
- Users ask many different kinds of questions.
- The system must combine information from several tools and adapt to intermediate results.
- Some variability is acceptable, and there are guardrails and human review for risky actions.

4.4 College Examples

Task	Best choice	Reason
Generate hall tickets for all registered students	Workflow	Fixed steps, must be exact every time
Answer "When is the library open?"	Chatbot (with the timings in its prompt)	Simple, single-turn question
Calculate internal marks from attendance and test scores	Workflow	Formula is fixed and must be auditable
"Help me pick electives that fit my timetable and interest in AI"	Agent	Needs timetable lookup, course lookup, and reasoning across results
Summarize each day's circulars and email them to staff	Workflow with an LLM step	Fixed sequence; LLM only summarizes
Investigate why a student's fee payment is not reflected	Agent (with human approval)	Unknown number of checks across systems

5. Worked Example: One Task, Three Systems

Scenario: a college has three courses with fees CS101 = Rs. 12,000, AI202 = Rs. 18,000, DS303 = Rs. 15,000. A student asks: **"What is the total fee for CS101 and AI202 after a 10% scholarship?"** The correct answer is $(12,000 + 18,000) \times 0.9 = \text{Rs. 27,000}$.

System	What happens	Result
Chatbot	The LLM has never seen the fee data, so it guesses numbers or asks the student for them.	Wrong or unhelpful; may sound confident
Workflow	Code extracts course codes with a pattern, looks up fees, detects the word "total" and the percentage, and computes the result.	Correct, but only for question phrasings the programmer anticipated
Agent	The LLM calls <code>get_course_fee("CS101")</code> , <code>get_course_fee("AI202")</code> , then <code>calculator("(12000+18000)*0.9")</code> , and writes the answer.	Correct, and can also handle new question types

You will run all three systems on your own machine in the Day 1 lab session.

6. Key Terms

Term	Meaning
Token	A small unit of text processed by an LLM.
Context window	The maximum amount of text (in tokens) the LLM can read in one call, including instructions, history, and tool results.
Knowledge cutoff	The date after which the model has no training data.
Hallucination	A fluent but false or unsupported output.
Tool	A function the agent can request the program to run, described to the LLM with a name, description, and parameters.
Tool call	A structured request from the LLM containing a tool name and arguments.
Observation	The result of a tool call, returned to the LLM.
Agent loop	The repeated cycle of reason, act, and observe until the task is complete.
Agency	The ability to act on an environment toward a goal.
Autonomy	The degree to which a system chooses its own actions without human direction.
Deterministic	Always producing the same output for the same input.

7. Further Reading

1. Anthropic, "Building effective agents" (workflows vs agents, and when to use each) <https://www.anthropic.com/engineering/building-effective-agents>
2. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR 2023 <https://arxiv.org/abs/2210.03629>

3. J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS 2022
<https://arxiv.org/abs/2201.11903>

Text book reading: Chip Huyen, AI Engineering (O'Reilly, 2025), chapter on RAG and Agents, sections introducing agents and tools.